Intelligent Systems with Applications 28 (2025) 200606

Contents lists available at ScienceDirect

Intelligent Systems with Applications

journal homepage: www.journals.elsevier.com/intelligent-systems-with-applications

Alert correlation for intelligent threat detection and response

Bronagh Lanigan [a,*], Zeinab Rezaeifar [b], Federico Cruciani [a], Michael Milliken [a], Jordan Vincent [a], Samuel Moore [a], Muhammad Aaqib [a], Alan Mills [b], Pushpinder K. Chouhan [c], Alfie Beard [c], Chris D. Nugent [a], Luke Chen [a], Alex Healing [c]

[a] Ulster University, School of Computing, Belfast, BT15 1AP, Northern Ireland, UK
[b] University of the West of England, Computer Science and Creative Technologies, Frenchay Campus, Bristol, BS16 1QY, UK
[c] British Telecommunication, 5 Lanyon Pl, Belfast, BT1 3BT, UK

ARTICLE INFO

Keywords:
Advanced Persistent Threat (APT)
Correlation engine
Intrusion Detection System (IDS)
Multi-step attack
Intrusion Response System (IRS)

ABSTRACT

With the increasing diversity of IoT devices, keeping IT systems secure is becoming increasingly difficult. Attackers exploit vulnerabilities within the system in order to access sensitive information, typically reaching their objective through several steps. Current Intrusion Detection Systems (IDSs) focus on low-level alerts, and tend to produce a high rate of false positives. This type of information alone is insufficient for the detection of sophisticated attack scenarios such Advanced Persistent Threats (APTs). Consequently, correlation techniques have recently been introduced to correlate alerts and reconstruct attack scenarios, however, various attack scenarios exist, with diverse characteristics. Also, different steps of the APTs scenarios may have their own characteristics. Therefore, finding a proper method that covers all cases remains a challenge. Moreover, after detecting APTs, how the system should respond to these attacks to avoid sabotage to the system remains a challenge. Thus, in this paper, first for detection of the attacks, we classify different cases, and then, a method based on different characteristics of attack patterns is proposed to detect APT scenarios. The proposed method consists of two main phases: APT detection and the intelligent hybrid response framework. In APT detection phase, similar alerts are aggregated and attack graphs are generated based on a similarity matrix. These graphs, combined with third party API data enable alert correlation and APT scenario detection. Entity graphs are then created to visualise host behaviour, and alert graphs are analysed to detect APT scenarios. In the response phase, attack graphs produced from the correlation inform the hybrid response framework, integrating knowledge and data-driven components that facilitate automated or recommended mitigation. The approach was evaluated on the ZeekData24 dataset. Obtained precision and recall on the malicious traffic was observed to be 96.65% and 87.04% respectively. The results show that our approach can effectively filter false positive alerts with a reduction of the data going from 10,063 alerts daily to 586 meta-alerts, pruned to 48 attack graphs and finally reduced to 20 suspicious attack graphs.

1. Introduction

In December 2021, a vulnerability affecting the Log4J logging service was discovered. The vulnerability allowed attackers to deploy malicious code on affected servers, rapidly leading to thousands of successful exploits that were used either for ransomware attacks or data exfiltration. More than 93 million attacks were reported in just a few days.[1] The massive scale of this incident caused a race against time in identifying vulnerable systems and updating their logging service to the latest software version. The attack affected major organisations worldwide, including governmental organisations such as the Belgium' s Ministry of Defence, and large enterprises. This was not an isolated incident. In August 2022, the NHS was targeted by a ransomware attack,[2] and similar episodes have been continuously reported in many countries in the last three years. Early identification of such an event is crucial in order to stop attacks before the victim hosts gets

* Corresponding author.
E-mail addresses: b.lanigan@ulster.ac.uk (B. Lanigan), zeinab.rezaifar@uwe.ac.uk (Z. Rezaeifar), f.cruciani@ulster.ac.uk (F. Cruciani), m.miliken@ulster.ac.uk (M. Milliken), jj.vincent@ulster.ac.uk (J. Vincent), s.moore2@ulster.ac.uk (S. Moore), m.aaqib@ulster.ac.uk (M. Aaqib), alan.mills@uwe.com (A. Mills), pushpinder.kaur.chouhan@bt.com (P.K. Chouhan), alfie.beard@bt.com (A. Beard), cd.nugent@ulster.ac.uk (C.D. Nugent), l.chen@ulster.ac.uk (L. Chen), alex.healing@bt.com (A. Healing).

[1] https://www.computerweekly.com/news/252512071/Top-three-questions-about-the-Log4j-vulnerability
[2] https://www.bbc.co.uk/news/technology-62506039

https://doi.org/10.1016/j.iswa.2025.200606
Received 17 September 2024; Received in revised form 9 September 2025; Accepted 6 November 2025
Available online 7 November 2025
2667-3053/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

compromised and the attack begins to spread affecting other hosts within the same network.

In order to detect security threats, IDSs are currently deployed to monitor network traffic, detect attacks, and attempt to stop them before the victim hosts are compromised. IDSs are software applications analysing network traffic and systems log files in order to generate alerts whenever suspicious activities are detected. Network security analysts can then intervene based on IDS-generated alerts by taking appropriate mitigation actions to stop the spread of the attack. IDSs, however, often produce vast amounts of data with a notably high rate of false positives. Additionally, alerts that potentially belong to the same attack might be flagged as separate incidents, complicating the identification of APTs which typically involve multiple steps. These challenges collectively render the early detection of such incidents akin to finding a needle in a haystack for security analysts (Elshoush & Osman, 2011).

1.1. Problem statement

Complex attacks, such as the Log4J case, require multiple steps to be performed before the attacker can take control over a victim system. The attackers may attempt to gain access to the system using social engineering methods such as watering holes and phishing emails, or through scanning the target host for known vulnerabilities. Upon successful intrusion, the attackers can then try to maintain their foothold by exploring the network, exploiting further vulnerabilities of the system, etc. Finally, they reach their goals by sabotaging the system, or using newly controlled machines to launch attacks on other victims, or trying to leak sensitive data (Lanoe, Hurfin, & Totel, 2019; Xosanavongsa, Totel, & Bettan, 2019a). These types of attacks may occur over a large window of time, and the events related to the same attack scenario may be scattered over different machines (Xosanavongsa et al., 2019a). IDSs can detect some of these stages and generate security alerts, however, these alerts are reported as isolated incidents, and the possible causal relationship between the alerts belonging to the same attack is currently beyond the ability of IDSs. This leads to the accurate identification of attack steps being obscured by thousands of false alarms or unrelated events (Lanoe et al., 2019; Navarro, Deruyver, & Parrend, 2018). Consequently, researchers have been exploring the possibility of developing alert correlation algorithms capable of analysing the large volume of generated alerts with the aim of:

- identifying groups of alerts attributable to the same attack.
- choosing an appropriate action as a countermeasure.

As with many other domains within the application of Artificial Intelligence (AI), the presence of data, either to train or to validate models, is an essential and a fundamental requirement. Internet security is no exception in this respect, where the lack of adequate datasets has been obstructing the application of AI as exemplified in the context of network IDS. When handling network security datasets, the two main approaches for dataset collection are packet-based and flow-based. Both approaches have their advantages and disadvantages, however, concerning scalability, flow-based approaches have been widely adopted due to the reduced size of collected data. Consequently, the research community has predominantly focused on flow-based methods [1].

1.2. Drawback of conventional methods

Existing alert correlation methods can be divided into two main groups: Knowledge Driven Approaches (KDA) and Data Driven Approaches (DDA). KDA include conventional rule-based approaches that are capable of detecting APTs using rules. The main limitation of KDA is that they can only detect previously observed situations for which domain experts have already codified these detection rules. With attackers constantly searching for new vulnerabilities to exploit, and new methods to carry out network attacks, which presents challenges in adapting to cover emerging threats, primarily due to the substantial domain expertise required for updating the rule base. DDAs leverage AI for the automatic categorisation and identification of alert traffic patterns, encompassing known attack types and detecting anomalies, such as unusual traffic, which may signify previously unseen potential threats. Contrary to KDA, DDA for threat detection can evolve over time with the presence of new data being collected with limited human oversight. However, on the other hand, these models are subject to the cold-start problem, i.e., they need a large volume of labelled data for initial training, in order to detect anomalies or to be able to recognise alerts consistent with attack patterns.

1.3. Objective

The objective of this paper was to study novel correlation methods enabling a Human in the Loop (HITL) approach in order to overcome limitations of both KDA and DDA. The method aims at correlating alerts by identifying possible connections between alerts that might belong to the same attack. Suspected attacks, identified using the developed correlation method, can then be analysed by a security analyst along with the proposed response as a countermeasure. The analyst can then review and confirm the proposed response, or select a different action. In the latter case, the analyst's final choice is stored and used to automatically propose the same action the next time these same circumstances re-occur.

1.4. Contributions

Contributions of this research are summarised as below.

1. A hybrid data and knowledge-driven approach is proposed to correlate alerts and detect APTs scenarios.
2. A synthetic dataset with real APTs scenarios are collected to evaluate the proposed method.
3. Preliminary experiments regarding the optimisation of the Correlation Engine have been presented.
4. Finally, a hybrid knowledge and data-driven response system is proposed to provide a timely action to alerts raised.

1.5. Paper structure

The remainder of this paper is organised as follows. Section 2 provides a concise review of related work for the detection of APT and then for APT response methods. Section 3 provides details on the proposed method for APT detection and the intelligent response framework. The novel dataset and the publicly available real-world dataset used for evaluating the proposed method are explained in Section 4. The evaluation methodology is described in Section 5. The correlation engine optimisation is detailed in Section 6, followed by the discussion in Section 7. Finally, Section 8 concludes the conceptual framework and introduces future work.

2. Related work

Several methods have been introduced to address detection and response to APT scenarios. This section provides an overview of state-of-the-art detection methods, then focuses on response methods for APT scenarios.

2.1. Reconstruction and detection APT methods

Several alert correlation methods addressing the attack reconstruction using alerts produced by IDSs have been proposed in recent years.

Alert correlation methods typically address a combination of elements from a common set of objectives:

1. alert verification (Meng, Li, & Kwok, 2015; Meng, Li, et al., 2012) - reducing the number of false alarms
2. alert prioritisation (Alsubhi, Al-Shaer, & Boutaba, 2008; Andrew & Kathrine, 2018)- identifying which alerts are more relevant
3. alert aggregation (Zhou, Leckie, & Karunasekera, 2010)- putting together groups of alerts that can be processed as a unique entity, thus reducing the number of elements to be processed
4. and finally, detection of causality between alerts to reconstruct APT attack scenarios, with the goal of predicting and preventing the next step of the attack.

DDA for correlating alerts, in particular when using Machine Learning (ML) methods, require a large amount of labelled data, and typically require a training phase, making them subject to the cold-start problem. This drawback comes with the advantage of data-driven approaches being able to evolve over time, learning on new data instances, e.g. in a HITL scenario in which a data analyst can provide new labels to adapt the model to detect a new type of attack scenario. On the other hand, KDA for correlating alerts, are immediately operative, or at least after the steps necessary to incorporate a priori expert knowledge into the system. These methods, however, are static and cannot evolve over time. In the following, some of the most recent examples of alert correlation methods that have been proposed within these macro-categories are provided.

KDA, such as rule-based (Alserhani, 2016; Godefroy, Totel, Hurfin, & Majorczyk, 2014; Lanoe et al., 2019; Ning, Cui, & Reeves, 2002; Qi et al., 2023) and scenario-based methods (Bateni, Baraani, Ghorbani, & Rezaei, 2013; Eckmann, Vigna, & Kemmerer, 2002; Hassan, Bates, & Marino, 2020; Hu et al., 2020; Park et al., 2023; Xiang, Dong, & Yu, 2005), apply a formal representation of expert knowledge to detect and reconstruct APTs scenarios. In rule-based approaches, for instance, expert knowledge is formalised as a set of rules that can be used to detect different attack scenarios based on a set of observed alerts. Several approaches have been proposed to define rules. Among these, a relevant group of rule-based methods use prerequisite and consequence of attack information as the main knowledge base for rule definition (Alserhani, 2016). Alert correlation in this case considers prerequisite conditions including, for instance, expected preliminary steps and/or known vulnerabilities. These methods can then correlate alerts by matching a set of new alerts with previous alerts corresponding to a prerequisite condition of the same attack pattern. Also, Qi et al. (2023) introduce an ontology-based method using a knowledge-based graph to create attack rules (Qi et al., 2023). The knowledge graph is used to analyse correlation in real-time data.

As an alternative to rule-based methods, scenario-based approaches use a representation of an attack scenario, and detect an attack whenever collected alerts match one of the known attack scenarios. Scenario-based approaches either use a formal language, such as LAMDBA (Cuppens & Ortalo, 2000) and STATL (Eckmann et al., 2002), to define attacks in the form of logic formulas targeting systems or use open-source tools such as MulVAl (Ou, Govindavajhala, Appel, et al., 2005) and adversary knowledge base models such as MITRE ATT&CK (Strom et al., 2018) to detect APT scenarios (Hassan et al., 2020; Hu et al., 2020; Park et al., 2023) or use data mining methods to identify attack scenarios (Xiang et al., 2005). The main disadvantage of knowledge-based approaches is that they rely solely on expert knowledge to reconstruct attack scenarios, and these methods cannot easily be adapted to detect unknown or new attack scenarios, nor cases in which the attacker operates unconventionally.

DDA, such as statistical analysis-based and similarity-based techniques, use information from data to detect APTs through anomaly detection or identify relations between alerts and reconstruct possibly ongoing attack scenarios. While the majority of DDA behave similarly to IDSs and use ML and Deep Learning (DL) approaches to distinguish APT (abnormal) and normal traffics (Do Xuan & Dao, 2021, 2021; Neuschmied et al., 2022; Sharma, Gupta, Singh, & Saraswat, 2023; Shen, Perigo, Curry, et al., 2023; Zimba, Chen, Wang, & Chishimba, 2020). There are some statistical analysis-based methods that use statistical information or ML/DL approaches to extract relationships between network traffic (Niu et al., 2022) or assess the relation between alerts (Qin & Lee, 2004; Shittu, Healing, Ghanea-Hercock, Bloomfield, & Rajarajan, 2015) for APT detection. For instance, in Qin and Lee (2004) the authors used Bayesian networks, and in Zhu and Ghorbani (2006), the authors used two different neural network methods for generating probability values in order to find which previous alerts can be related to the current set of alerts. Similarity-based approaches such as (Julisch, 2001; Shittu, Healing, Ghanea-Hercock, Bloomfield, & Muttukrishnan, 2014; Shittu et al., 2015) use similarity functions and information of alert fields to assess the degree of connection between alerts. Attack scenarios can then be generated on the basis of set of alerts exhibiting a strong connection. Also, there are different DDAs based on the provenance graph to detect cyber-attacks (Irshad et al., 2021; Lv et al., 2022). In these methods, the provenance graph uses the graph structure to reveal the process of threats. In the provenance graph, the nodes include system entities such as processes and files, and the edges include the process between entities such as reading and opening files. The provenance graph can provide causality between events and enrich them with contextual information which helps analysts to investigate attacks. However, finding a suitable graph which is well built and can manage storage and data remains a challenge. Also, with APT scenarios which can happen over a long period of time, for example due to long periods of dormant or minimal activity. As such, the provenance graph size will keep increasing over time which makes efficient storage and real time detection difficult (Lv et al., 2022).

As previously mentioned, the main advantage of DDA is that they can detect unknown and known attack patterns without using expert knowledge. These methods, however, tend to generate a higher rate of false positives if an appropriate countermeasure is not taken (Moore, Cruciani, Nugent, Zhang, Cleland, & Sani, 2023). For instance, finding an appropriate similarity function that covers all cases of different attack patterns is quite a challenging task, as each attack pattern can present its own features and characteristics and an over-sensitive similarity function can result in connecting alerts that in reality do not belong to the same attack. Another limitation is that similarity based methods often consider IP addresses to evaluate the similarity between two alerts, however, attackers can spoof their IP address to increase the chances of performing a structured attack without being detected.

Hybrid approaches such as (Barzegar & Shajari, 2018; Ren, Stakhanova, & Ghorbani, 2010; Saad & Traore, 2013; Wang, Guo, & Fang, 2022; Xosanavongsa, Totel, & Bettan, 2019b) combine the use of expert knowledge to detect known attack patterns and DDA, such as similarity and analysis-based methods, to detect unknown attack patterns. In Wang et al. (2022), the ontology-based method is introduced to correlate alerts and security logs to detect APT scenarios. This method is adopted to edge computing architecture in which the edge servers collected alerts and parse them based on the ontology model then the collected alerts will be sent to Security Collections Center (SCC) for analysing. Redundant alerts will be removed, and the remaining alerts correlated based on their target and source within the set time threshold. Finally, alerts will be transformed to graph structure, and Monte Carlo Tree Search (MCTS) (Kocsis & Szepesvári, 2006), which is an algorithm to find an optimal solution in a given domain, is applied to detect missed steps from edge servers and log files. In this method, the APTs will be presented in the form of the alert graph. In Khosravi and Ladani (2020), the authors defined a hybrid approach to detect APT scenarios on each host. The model is based on tracking Intrusion Kill Chain (IKC)(Assante & Lee, 2015). In this method the alerts generated from Security Information and Event Management (SIEM)(González-Granadillo, González-Zarzosa, & Diaz,

2021) on each host separately are considered. Then, the alerts are labelled based on alert type and IKC steps to generate meta-alerts. After that the casual relation between alerts based on cause and effect in IKC steps is identified. Finally, the host will be scored for being infected by APT based on number of IKC steps detected and the duration that this IKC has been created. However, since this method focuses on the host instead of whole system, it may fail to capture the whole attack scenario that involves different hosts (victims) to attack a target, such as infiltration attacks. Another hybrid approach is CONAN (Xiong et al., 2022) which is the contextual based method to detect APTs. This method uses control and data flow to explain contextual behaviours. In this method, firstly the stated-based framework is introduced. The state in this method is the semantic data, and the framework only keeps the state of the current processes and files for detecting APT. By using predefined rules, all events and logs are processed to alter the states of the processes. After changing the process to malicious state, the attack will be detected. Finally, the attack graph which indicate how the attack happens will be created. In Saad and Traore (2013), the reconstruction attack scenario methods are classified into two parts: clustering approaches and mining approaches, and knowledge-based approaches. In this method, the authors used similarity techniques to cluster alerts and apply association rule mining to define order between alerts in the specified time interval. To handle missing attack steps or false negatives, the approach used environmental knowledge along with the attack scenario graph to identify missing steps that were not detected by IDSs. However, for the missing attack stage, the method needs to investigate many possible cases and requires significant environmental knowledge (for instance, on known vulnerabilities). In Ren et al. (2010), the authors proposed a method that uses the Bayesian model to extract causality features between alerts without expert knowledge. Based on these features, the proposed method predicts the next step of the attack in real-time. In other words, the method combines the use of expert knowledge with inference approaches in order to detect APTs automatically.

Summarising the state of the art of alert correlation, the detection of APTs through the examination of alerts from multiple IDSs systems remains an open challenge. Within this landscape, hybrid solutions emerge as the most promising, leveraging the strengths of both approaches.

2.2. APT response methods

In contrast to IDSs, Intrusion Response Systems (IRSs) have gathered much less research focus over time due to the inherent risks associated with incorrect automated responses and scarcity of available datasets for testing IRSs. However, recently research into IRS has begun to emerge at a rapid pace (Gupta, Jindal, & Bedi, 2023).

Intrusion Response is the process of deploying an action that prevents, mitigates or stops an attack. There are two commonly reviewed techniques within IRSs which include passive and active IRSs (Inayat, Gani, Anuar, Khan, & Anwar, 2016). Passive IRSs, also known as non-combative IRSs, inform security experts of the alerts, but do not perform any actions automatically. Passive responses do not impact the network or host performance, for example, collecting logs, generating alerts, or aggregating information from 3rd party tools to observe more information on an attacker (Anuar, Papadaki, Furnell, & Clarke, 2010). However, passive actions are insufficient when responding to rapid attacks or vast numbers of attacks due to their non-active nature, therefore they are unsuitable for use in an enterprise network or on sensor networks typically found within the Internet of Things domains. Conversely, Active IRSs, also described as combative actions, automatically respond to alerts without human intervention. Such actions impact the performance of the network or a host in some way, for example, blocking an IP address, updating a firewall rule or terminating a user session (Mateos, Villagrá, Romero, & Berrocal). It is crucial for an IRSs to evaluate the cost of an attack to the system, to facilitate the appropriate decision to deploy active or passive actions (Chouhan, Quigley, Beard, & Chen, 2021).

Based on available alert information, KDAs or DDAs can be used to build an active IRSs. KDAs use past experiences by matching current alerts to relevant previous alerts, therefore guiding decision-making in IRSs. This means the outcome for an alert can be implemented and evaluated effortlessly. While KDAs are effective for responding to known attacks, they fail to adapt to emerging threats or handle APTs, therefore, may prove ineffective against the latest sophisticated and constantly evolving threats, prompting a trend towards DDAs within IRSs (Iannucci & Abdelwahed, 2018). DDAs involve analytical processes that leverage network data to predict and enhance responses to various attacks through training ML models on various previously known attack scenarios, enabling automated predictions, decisions and responses, for different attack scenarios. However, DDAs face limitations in testing datasets and real-time evaluation for development of data-driven real-time IRSs.

In latest developments for IRS, DDAs have been explored. For instance, Iannucci and Abdelwahed propose a cost-sensitive IRSs using the Markov Decision Process framework for long term planning. A threshold with attack probabilities is leveraged to prevent disruptive responses, unless the system is under imminent attack and has no other options (Iannucci & Abdelwahed, 2018). An adaptive cost-based IRS is presented by Nejat and Kabiri, which considers the attack cost in terms of attack severity, type of attack and the importance of the targeted system in prioritising responses. The system adapts the responses and utilises a feedback mechanism to assess their outcomes (Nejat & Kabiri, 2017). In a similar study, a cloud IRS has been proposed by Kholidy, et al. which monitors and evaluates system events for risks. Once the risk is determined, an appropriate countermeasure, or response, is selected to mitigate the attack on the cloud services (Kholidy, Erradi, Abdelwahed, & Baiardi, 2016). However, this study fails consider the number of vulnerabilities and connections into account, when prioritising alerts (Hassan et al., 2020). More recently, reinforcement learning and deep learning approaches have emerged to tackle open issues in IRS. N. Hersen utilises fuzzy logic to investigate requirements required for building effective IRSs. It was found that uncertainty in IDS predictions is crucial for effective IRS implementations (Hersén, Hacks, & Fögen, 2021). Similarly, H.A Kholidy evaluates potential damage of an attack and response, in terms of cost associated with both the attack and response selected to mitigate this. Responses use fuzzy inference techniques to monitor the level of risk associated with a response before deployment (Kholidy, 2021). Finally, authors in Iannucci, Barba, Cardellini, and Banicescu (2019) demonstrate effectiveness and scalability of both IDSs and IRSs in complex environments through the use of reinforcement learning. A KDA is deployed for the detection of attacks and evaluation of proposed automated responses, prototyped on a Virtual Machine (VM). Whilst experimental results prove effectiveness of responses with the potential to scale to large environments, the approach only used known attacks and fails to optimise the algorithmic complexity of responses when summed together.

The following section describes the proposed data-driven method for alert detection and correlation. We apply the expert knowledge to analyse reconstructed attack graphs and entity graphs, to detect and respond to APT scenarios, utilising the hybrid response framework to mitigate threats and deploy actions.

3. Proposed method

The proposed solution is composed of two parts: the Advanced Threat Detection, and the Intelligent Hybrid Response Framework described in the next sections.

3.1. Advanced threat detection

3.1.1. Situation assessment

The main role of an IDS is to generate alerts when events associated to a potentially malicious attack are detected. A variety of methods can be utilised to produce these notifications, such as examining system log files and analysing statistics on network traffic, where each notification is linked to a threat category. Complex attack scenarios frequently comprise multiple stages. Initial stages often involve preliminary reconnaissance, during which attackers attempt to identify vulnerable services on hosts within the targeted network. For example, a reconnaissance phase may contain a port scan used to determine the services running on a specific target host. Following this initial stage, attackers can attempt to establish communication with the target host for further information collection, such as details about software versions.

After completion of the reconnaissance phase, a range of techniques can be deployed by the attacker, based on their goals. Attack objectives can vary significantly; for instance, in a Denial of Service (DoS) style attack, the attacker may seek to disrupt services provided by the target system. In other cases, such as brute force attacks, the attackers may try to gain control of specific services or hosts, where different username and password combinations are attempted to gain access. Another case is ransomware attacks, which have become increasingly prevalent. In this style of attack, the attacker encrypts data on the victim host, rendering it unusable for the owner and demanding payment to decrypt the data.

Despite the diversity of attack patterns, IDSs can only detect specific types of threats and are unable to discern causal relationships between seemingly unrelated events, even if they are part of the same attack scenario (Elshoush & Osman, 2011). This relationship can only be established in the alert correlation phase. Alert correlation involves various methods for analysing IDS alerts to identify which subset of these alerts may be linked to the same attack, which was first introduced in previous works (Mirheidari, Arshad, & Jalili, 2013; Ramaki, Amini, & Ebrahimi Atani, 2015). There are two key objectives in alert correlation: decreasing the false positive rate and connecting events to established attack patterns. Subsequently, an IRS can then utilise this knowledge to anticipate the next phases of an ongoing attack and implement appropriate countermeasures to mitigate additional harm to the targeted system. Our proposed alert correlation method consists of two phases:

1. Alert aggregation: This phase involves grouping similar or identical alerts that can be treated as a single entity, often referred to as 'meta-alerts.' While this step does not identify causal relationships, it reduces the volume of alerts for processing.
2. Alert correlation: In this phase, relationships between aggregated alerts (meta-alerts) are identified, which may be part of the same APT scenario.

Fig. 1 provides an overview of the entire correlation process, where the input data is derived from an IDS and a Domain Generated Algorithms (DGA) detector (Cruciani, Moore, Quigley, Nugent, & Sani, 2022). Network traffic is monitored through an IDSs, to detect common attacks such as Brute Force, Ping Scan, Port Scan and DoS, while the DGA module analyze Domain Name Service (DNS) requests to generate alerts when suspicious domain names, possibly indicative of botnet attacks, are detected. In the third-party API module, information about potential attackers, including IP addresses or domain names, is cross-referenced with data from various external sources to estimate the attacker's reputation score. The accumulated information from the attack graph reconstruction and the third-party API module is leveraged in the correlation engine to analyse attack graphs and entity graphs, aiding analysts in detecting APTs. The entity graph illustrates the connections between the source and destination of generated alerts, providing a clearer picture for analysts to interpret attack scenarios. Finally, the hybrid response model takes the information surrounding the APT and proposes a response for mitigation purposes.

3.1.2. Alert aggregation

Alerts with common characteristics can be aggregated into groups of meta-alerts to reduce redundancy in attack graphs and streamline the analysis workload. Alerts featuring identical attack types alongside source IP addresses, destination IP addresses and destination ports, are grouped into a single meta-alert.

A meta-alert is a representation of a collection of alerts that are almost identical, except their associated timestamps. The timestamps are used to capture repetition of the same alert occurring in rapid succession as a unique item. If the time difference between two consecutive alerts with the same meta-data is less than a threshold T, the alerts will be merged into a single meta-alert. If the time difference exceeds the threshold, the alerts can still be considered as belonging to two separate meta-alerts.

3.1.3. Similarity function

Following the alert aggregation process, a similarity function compares available information on meta-alerts to assess the connections between them. Each meta-alert includes information on the source hosts, such as the presumed attacker(s), and destination hosts, i.e., the potential targets/victims of the attack, gathered from the available meta-data on the two hosts (source and target), including IP addresses, available network information (ISP/organisation), associated domain names, Media Access Control Name Service (MAC) addresses, communication port and available geographical information (estimated location associated with the IP). Specific fields of the meta-data may be more or less relevant than others depending on the type of attack to be detected, for instance, the similarity between meta-alerts on their destination hosts is useful for capturing coordinated attacks involving multiple attackers targeting the same destination host, e.g., in Distributed Denial of Service (DDoS) attack. In comparison, it is typical in APTs for attackers to attempt to perform various attack steps on the same target systems to accomplish their goal, therefore the source host and destination host similarity is beneficial for identifying these same attacker(s) in this scenario. Moreover, the similarity between the destination host of a meta-alert with the source hosts of later meta-alerts can be used to detect situations when the attack is spreading. For instance, during the propagation of an attack, e.g., ransomware, a successfully attacked victim may start acting as an attacker.

Besides source–destination similarity, timestamps play an essential role in the correlation of alerts. For the context of this paper timestamps are utilised and referred to as events occurring before or after, to serve as a basis for interpreting relevance for a security analyst. For instance, a timestamp may indicate when an attack occurred, when it was detected by a log, or when it was reported. Due to variations in latency across different IDS, the same attack may be recorded with different timestamps. Consequently, time is not employed strictly in a chronological or time-series analysis but rather as a measure of relevance. For example, if IDS1 records an attack one minute earlier than IDS2, the similarity measure remains high. Thus, temporal in this context signifies the relative ordering of attacks, enabling analysts to comprehend and respond effectively to security incidents. With this in mind, timestamp information helps detect causality between events, such as spotting a reconnaissance step just before an intrusion attempt. Time proximity can also be used for identifying coordinated attacks, such as DDoS, where multiple hosts act simultaneously against single target. The time similarity function utilises meta-alert timestamps to identify additional correlations. Time similarity is combined with the similarity between source and destination hosts to establish the final similarity for each pair of meta-alerts. Finally, weaker connections are pruned using a specified threshold, resulting in individual sets of connected alerts (attack graphs) that can be analysed in search of attack patterns. Four different perspectives based on (Haas & Fischer, 2019) are used to develop the final similarity calculation, each designed to capture all possible types of attack, computed simultaneously. The four categories for attack scenarios are summarised below:

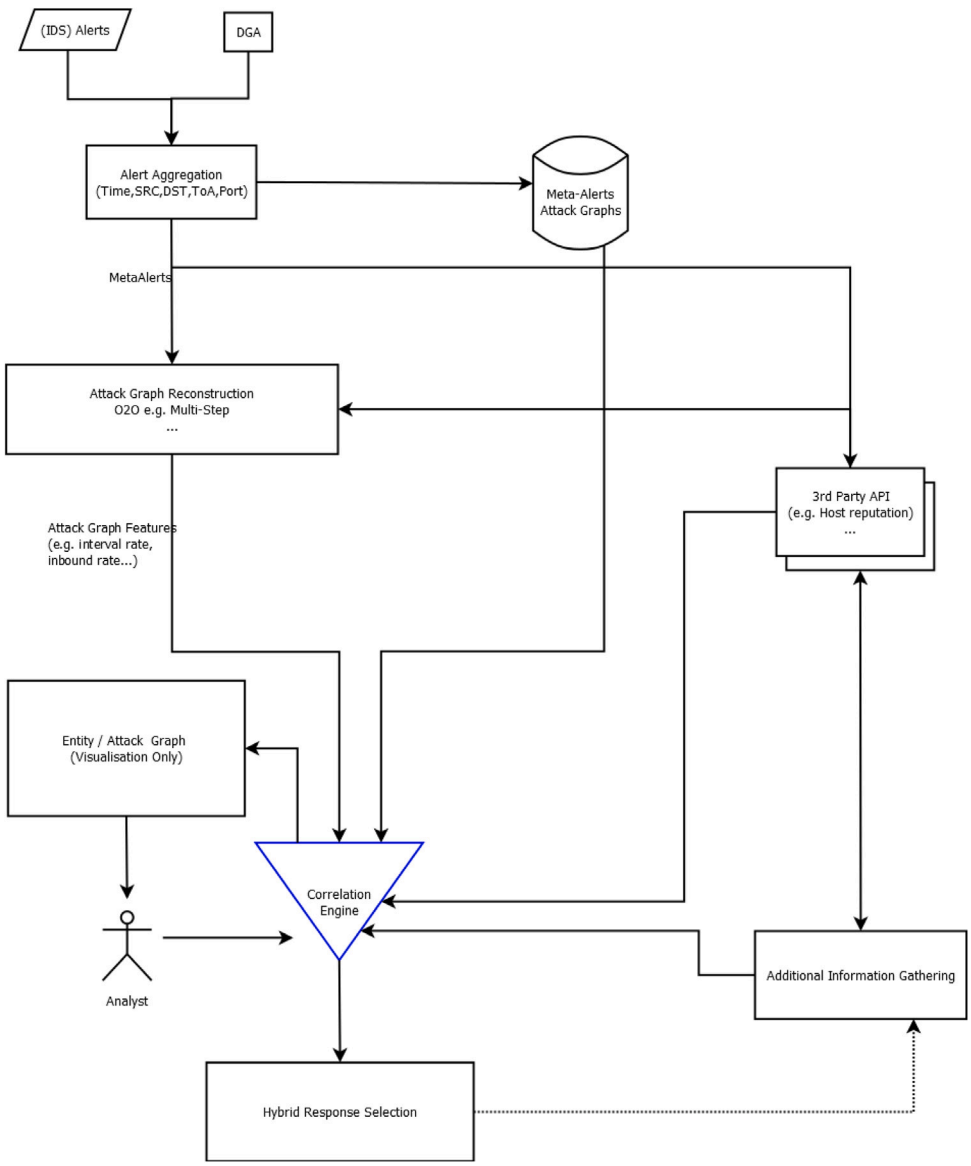

Fig. 1. Proposed process for Alert Correlation.

- One2one (O2O): In this case, there is one attacker and one victim, as for instance in brute-force attacks. Attacks in this group are characterised by having the same source and the same destination.
- One2Many (O2M): In this scenario, one attacker attacks many victims, as in the case of scanning a subnet, or as in the case of zombie attacks where the attacker tries to gain access multiple victims. In this case the O2M is the first step required to get control over the victims and then redirect them to attack the true target in a Many2One case. Multiple alerts with the same source IP address and different destination IP addresses can be categorised in this group.
- Many2One (M2O): In this scenario, several attackers try to attack one target, as for instance in DDoS attacks. In this case, the attackers try to target one service in the target system so the alerts related to this scenario will have the same destination IP address (and typically the same port unless the attackers are trying to perform a DoS on more the one service on the same machine).
- Many2Many (M2M): In this case, multiple attackers target multiple victims such as in worm attacks. In this scenario, as attackers target a specific service, the alerts should have at least the same destination port as a common characteristic.

The similarity functions for these four perspectives combine time and host similarity with different weights in order to capture different types of attacks. These weights can either be static or user-configured to allow analysts to set a proper value based on their observations. The details of calculating each parameter in the similarity function are explained as below:

1. The similarity of source IP addresses: the similarity between the source IP address of two alerts is used to verify if different alerts are coming from the same source, or from similar sources (e.g. IP addresses in the same network). The similarity between two source IP addresses is defined

$$Ssim_{1,2} = \begin{cases} 1 & \text{if } srcIP_1 = srcIP_2 \\ 0 & \text{otherwise,} \end{cases} \tag{1}$$

   Where $srcIP_1$, $srcIP_2$ are source IP addresses of alerts 1 and 2 respectively.
2. The similarity of destination IP addresses: Similarity of the destination IP addresses of two alerts shows that the two alerts are targeting the same host, or two hosts in the same network. As in the case of source IP address, the similarity between two

destination IP addresses is calculated by using (2).

$$Dsim_{1,2} = \begin{cases} 1 & \text{if } dstIP_1 = dstIP_2 \\ 0 & \text{otherwise,} \end{cases} \tag{2}$$

Where $dstIP_1$, $dstIP_2$ are destination IP addresses of alerts 1 and 2 respectively.

3. The similarity between the source and the destination IP addresses of two alerts is a useful metric to highlight infiltration cases. In such cases, the same IP address observed to be the destination of an alert (i.e. the victim) can become the source (attacker) at a later time. Since in many cases, an attacker tries to compromise a host and through that host, propagate the attack towards another target. To calculate the similarity of this characteristic, the complete match between IP addresses is considered. Therefore if the source of one alert is completely similar to the destination of another alert, this characteristic will be 1, otherwise, it will become 0 as shown in (3).

$$Rsim_{1,2} = \begin{cases} 1 & \text{if } srcIP_1 = dstIP_2 \vee srcIP_2 = dstIP_1 \\ 0 & \text{otherwise,} \end{cases} \tag{3}$$

Where $srcIP_1$, $dstIP_1$, $srcIP_2$, $dstIP_2$ are source and destination IP addresses of alerts 1 and 2 respectively.

4. The similarity of the destination port is useful to identify alerts targeting the same service in the victim machine. Usually, an attacker tries to exploit the vulnerability of the service that typically will be running on standard ports (e.g. SSH on port 22, HTTP on 80, HTTPS on 443). The value of this characteristic has the value 1 or 0. If two destination ports are identical, the similarity of destination ports characteristic becomes 1, otherwise, it will become 0:

$$Psim_{1,2} = \begin{cases} 1 & \text{if } (dstPort_1 = dstPort_2) \\ 0 & \text{otherwise,} \end{cases} \tag{4}$$

Where $dstPort_1$ and $dstPort_2$ are destination ports of alert one and two respectively.

5. The similarity of time ranges of aggregated alerts is shown the relation of alerts' occurrences. Therefore, this parameter can be very useful for scenarios such as DDoS attacks where strong relation of times is an important factor for attackers to reach their goal. Also, it can be used to infer causality of alerts (attack steps) in the alert graph. To calculate time similarity, let two aggregated alerts A_1 and A_2 have time ranges (t_{1start}, t_{1end}) and (t_{2start}, t_{2end}) respectively. If A_2 precedes A_1 $t_{1end} > t_{2start}$, the exponential function is used to calculate time similarity. The exponential function is near to zero when time gap between two aggregated alerts are high and near one when the their time gap goes to zero. In the case that A_2 meets or overlaps A_1, the time similarity becomes 1:

$$Tsim_{1,2} = \begin{cases} e^{-(t_{1end} - t_{2start})/w_t} & \text{if } (t_{1end} > t_{2start}) \\ 1 & \text{otherwise,} \end{cases} \tag{5}$$

where w_t is time windows that is used to normalise alerts times.

The final similarity between two alerts are calculated in (6).

$$Fsim = (W_1 Rsim \vee (W_2 Ssim + W_3 Dsim)) + W_4 Psim + W_5 Tsim \tag{6}$$

Where W_1, W_2, W_3, W_4, W_5 are weights for $Rsim$, $Ssim$, $Dsim$, $Psim$, and $Tsim$ parameters respectively. In (6), when $Rsim$ is one, $Ssim$ and $Dsim$ will be zero, and when either of $Ssim$ or $Dsim$ will be one, $Rsim$ will be zero. The summation of weights consider in the final similarity should be one, i.e., $(W_1 + W_4 + W_5 = 1)$ or $(W_2 + W_3 + W_4 + W_5 = 1)$. Therefore, $W_1 = W_2 + W_3$.

The weights are configurable parameters in order to allow an analyst to set an appropriate value according to the observation situation and four cases of One to One (O2O), One to Many (O2M), Many to One (M2O), and Many to Many (M2M). For example, in the O2O case, the similarity between the source and destination IP addresses are more important features than the similarity between destination ports, since in some cases the same attacker may be targeting more than one service. As such, in the final similarity function for the O2O case, the final similarity value is calculated as a weighted average of the similarity calculated between IP and destination port, with a higher weight for IP similarity. Also, the similarity matrix in the M2O case, is based on the similarity of destination IP addresses and destination port. Since this scenario is designed to cover the case in which multiple attackers try to attack the same target.

Finally, a similarity matrix is defined for each case, therefore, each case can be analysed separately. The result of these separate analyses can then be combined in an attempt at reconstructing a view of more complex attack scenarios. For each case a similarity function was defined with the goal of assessing similarity between meta-alerts. The application of the function on a set of meta-alerts results in the calculation of an NxN similarity matrix, where N is the number of meta-alerts.

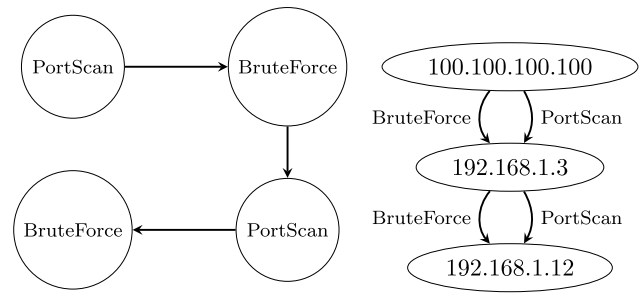

Fig. 2. Attack graph for an infiltration attack (left) showing the steps of the attack and the corresponding entity graph (right) which shows the chain of attacks and the hosts involved.

3.1.4. Attack graph reconstruction

An attack graph is a directed graph $G = (V, E, W)$, where each vertex $v \in V$ corresponds to an aggregated meta-alert whose edge $e_i \in E$ shows the connection between a pair of meta-alerts i and j, and $w_{i,j} \in W$ represents the degree of connection defined by the similarity matrix between i and j. In other words, an attack graph comprises a collection of nodes, representing meta-alerts assumed to pertain to the same attack, connected by a set of edges that describe the sequence of steps in an APTs. Once a similarity matrix for a set of alerts has been computed, the degree of connection (or similarity) between each pair of meta-alerts is established, enabling the identification of potential attack graphs. During this phase, a predefined threshold is employed to remove weaker connections, with the aim of retaining only sets of meta-alerts displaying robust mutual connections. These pruned, interconnected groups of meta-alerts correspond to the identified attack graphs. The edges within the attack graph delineate the chronological order of the attack, utilising time relationships to sort the attacks.

Fig. 2 serves as an illustration of an attack graph related to an infiltration attack. In this instance, the attacker gains control of the victim system after executing a port scan and multiple brute force attempts. Subsequently, the attacker initiates another port scan and endeavours to launch a brute force attack on another host within the target network. In our proposed approach, the edges between alerts are directional, and their direction, along with considerations for APTs where specific events must occur in a specific sequence, are based on the time intervals of the two interconnected meta-alerts.

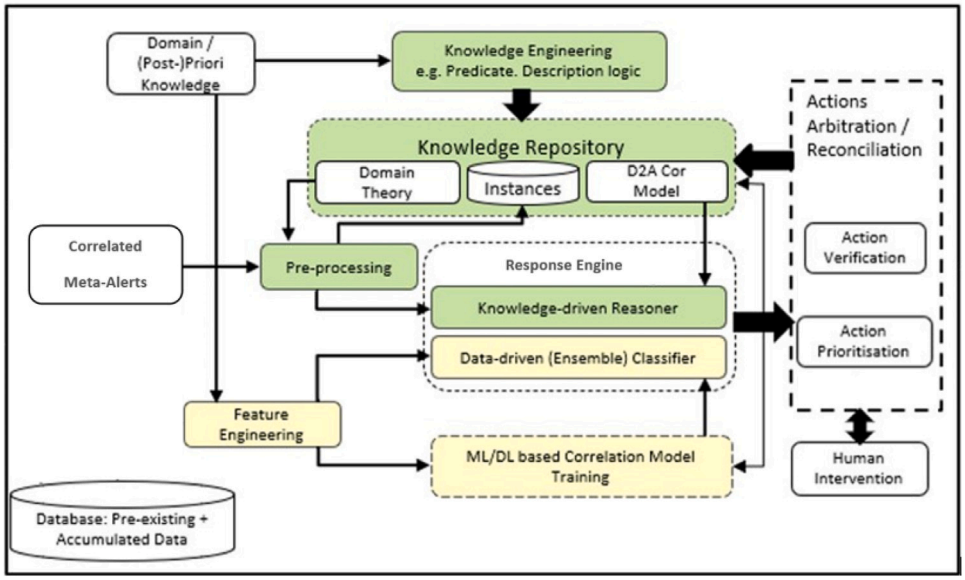

Fig. 3. Hybrid Knowledge and Data Driven Response Diagram.

3.1.5. Entity graph reconstruction

Entity graphs offer an explicit representation of the associated hosts as a visualisation. When used in conjunction with the attack graph visualisation, both visuals help analysts to interpret the detected attack patterns. The entity graph is a directed graph $\acute{G} = (\acute{V}, \acute{E}, \acute{W})$, where each vertex $\acute{v} \in \acute{V}$ corresponds to either source or destination IP address of the aggregated alert in the attack graph. Each edge $\acute{e} \in \acute{E}$ shows the connection between source and destination IP addresses of the aggregated alert in the attack graph, and $\acute{w} \in \acute{W}$ demonstrates corresponding the type of attack for that aggregated alert. Fig. 2, illustrates the entity graph corresponding to the attack graph in Fig. 2. The visualisation of entities (with each host identified by the corresponding IP address) illustrates the attacker host (with IP address 100.100.100.100) compromising the host with IP address of 192.168.1.3 and the subsequent attack spread to 192.168.1.12.

After the steps of alert detection, correlation and aggregation, the next challenge is responding to raised alerts appropriately. A framework was developed within which there are mechanisms for both automated responses and response recommendations for raised alerts, which were distinguished based on the need for a HITL before deployment, as a mitigation step. The automated response module consists of a hybrid knowledge and data-driven architecture as shown in Fig. 3. A hybrid approach was chosen to leverage the strengths of both knowledge-driven and data-driven methods while mitigating their individual weaknesses. The KDA allows the specialist analysts to add their domain knowledge of their specific network details to the system, whereas the DDA combines the information learned from the wide-ranging generic attacks seen during its training. The following section describes the hybrid response framework in more detail.

3.2. Intelligent hybrid response framework

The hybrid response framework comprises of two main components, the Knowledge Driven component and the Data Driven component. Together, these components make the hybrid response model, where each model can be utilised to deliver appropriate responses to incoming attacks from the correlation engine.

3.2.1. Knowledge driven component

The knowledge-based system leverages available information about the network systems and expert knowledge to apply responses to known attack scenarios. Weights are assigned to attacks based on their impact on confidentiality, integrity, and availability. For example, DDoS attacks primarily affect availability, brute force attacks can compromise both confidentiality and integrity, while ping scans may expose network information, impacting confidentiality and potentially availability. Network hosts and services are also categorised and weighted based on their importance. These values are correlated with attack effects to calculate a score, determining appropriate responses. Tests are performed to assess and verify feasibility of chosen responses. Response data is stored for future data mining. More details on the knowledge-based system can be found in Chouhan et al. (2021) (Chouhan, Quigley, Beard, & Chen, 2023).

3.2.2. Data driven component

The Data Driven model comprises the CICIDs 2017 dataset and Calibrated Random Forest (CRF) to build and evaluate a cost sensitive Intrusion Detection and Response System. The proposed CRF quantifies uncertainty in attack predictions through probability distributions. Subsequently, the predicted probabilities are used as confidence scores and integrated with domain expert knowledge for decision making in the IRS. Security analysts have the ability to manage responses deployed in the IRS through an adjustable threshold, ensuring active responses are only deployed when the IDS confidence is high, whereas less confident predictions require a HITL to oversee appropriate actions. Experimental results show this method enhanced cost-sensitivity in intrusion responses, managing alerts appropriately whilst effectively explaining model behaviour. Further details on the data-driven IRS can be found in Hussain et al. (2022).

4. Datasets for evaluation

In this section, the synthetic dataset and its collection process are first described. Then, the ZeekData24 dataset (Elam, Mink, Bagui, Plenkers, & Bagui, 2025), an openly available real-world dataset used for evaluating the proposed method, is explained. To the best of our knowledge, this is the only APT-focused public dataset that meets the requirement of capturing data from external network activity without relying on host or end-user information such as logs.

4.1. Data collection of synthetic dataset

To evaluate the correlation engine framework, several public datasets were identified, such as WISENT-CIDDS and CICIDS 17.

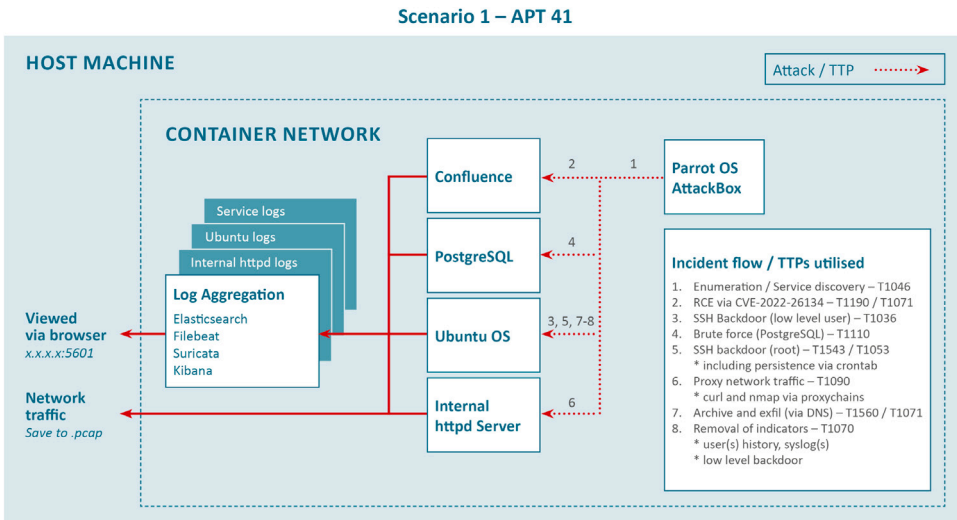

Fig. 4. Attack Scenario 1 of Synthetic Dataset.

However, these datasets contained limitations making them impractical to use for the correlation engine as it requires both input from multiple IDSs and cases of APTs. For example, WISENET-CIDDS does not provide APTs and CICIDS2017 contains APTs but for a reduced list of attackers. To ameliorate these concerns a synthetic dataset was designed to cover more suitable scenarios. The data was generated using *GoibhniUWE*[3] a container based cyber range used to create different services, which includes functionality for the generation of random traffic and noise, along with different attack traffic. The dataset was created through the aggregation of various different system logs and Packet Capture (PCAP) files for each scenario. To give a more detailed understanding of the dataset, three scenarios from the dataset are described next, APT-41, APT-28, and Turla.

4.1.1. APT-41 attack scenario

Fig. 4 shows details of the first attack scenario, APT-41, where a group of eight different attack steps were deployed against four target services, namely, Confluence, PostgresSQL, Ubuntu OS and an Internal httpd server. Two attacks were carried out against the confluence service, namely, service discovery and RCE via CVE. Brute force attacks were deployed against the PostgresSQL server, whilst an SSH backdoor (root and low level user) was deployed on the Ubuntu OS and used to archive target data, and remove indicators of activity. Proxy network traffic was deployed against the internal httpd server (from the Ubuntu OS) and all gathered archived data was exfiltrated using DNS. Service logs, Ubuntu logs and internal hhtpd logs were aggregated together using tools such as Elasticsearch, Filebeat, Suricata and Kibana and viewed in a browser for analysis. Network traffic was also collected throughout the entire attack as PCAP files for a closer analysis.

4.1.2. APT-28 attack scenario

APT-28 contains a five stageIKC deployed against two containerised services, Tomcat and Alpine OS. Attacks such as service discovery, targeted password guessing and malicious file uploads were carried out on Tomcat whilst ingress tool transfer and actions on target (including exfiltration), such as shell commands, process logging and keyboard logging were targeted on the Alpine OS. Similarly to APT-41 scenario, service logs and Alpine logs were collected and log aggregation of Elasticsearch, Filebeat, Suricata and Kibana were used for analysis, alongside PCAP file captures of the entire attack.

[3] https://github.com/uwe-cyber/GoibhniUWE

Table 1
Table of alert class, type, and meta-alert type mappings.

Alert class	Alert types	Meta-alert types
A1	ET SCAN Possible Nmap User-Agent Observed SURICATA ICMPv4 unknown code	Reconnaissance & Delivery
A2	HTTP-400 Bad Request	Exploitation
A3	ET POLICY Outgoing Basic Auth Base64 HTTP Password detected unencrypted HTTP-401 Unauthorised	Operation
A4	HTTP-200 Deploy	Data Collection & Exfiltration

4.1.3. Turla attack scenario

The Turla attack scenario comprised of four different services, namely, WordPress, Ubuntu OS, DNSMasq, and an Internal Open SMPTD Server. Nine different attack steps were deployed against these services. These attacks include service discovery and SQLI carried out against WordPress, local accounts. Ingress tool transfer, backdoor traffic signalling and service discovery via python and removal of indicators were all deployed against the Ubuntu OS, whilst DNS records were altered alongside user history, with syslogs and DNS changes reverted on the DNSMasq. Finally, exfiltration via SMTPD was carried out using the Internal Open SMTPD Server. Service logs, Ubuntu logs, Open SMPTD, DNSMasq and an aggregation of previously mentioned log analysis tools were all used to analyse the attacks. Like the other attack scenarios, network traffic was captured throughout the attack as a PCAP file.

4.2. Public real-world dataset

With the goal of creating a realistic modern dataset for benchmarking, the University of West Florida (UWF) generated a dataset from MITRE framework, using Zeek to generate detailed logs. The dataset is comprised of real network data created through UWF's Cyber Wargaming course. Using the same labs that were used for the previous UWF22 dataset, but with scripts to automate attacks. Attacks are directed against the 143.88.1.1–21 range. The following MITRE tactics are present in the data: Reconnaissance, Discovery, Credential Access, Privilege Escalation, Exfiltration, Initial Access, and Persistence. The overall scenario playing out through the dataset is that attackers use active Reconnaissance to scan network, successfully phish an employee for access, and then scan for additional attack vectors. Further

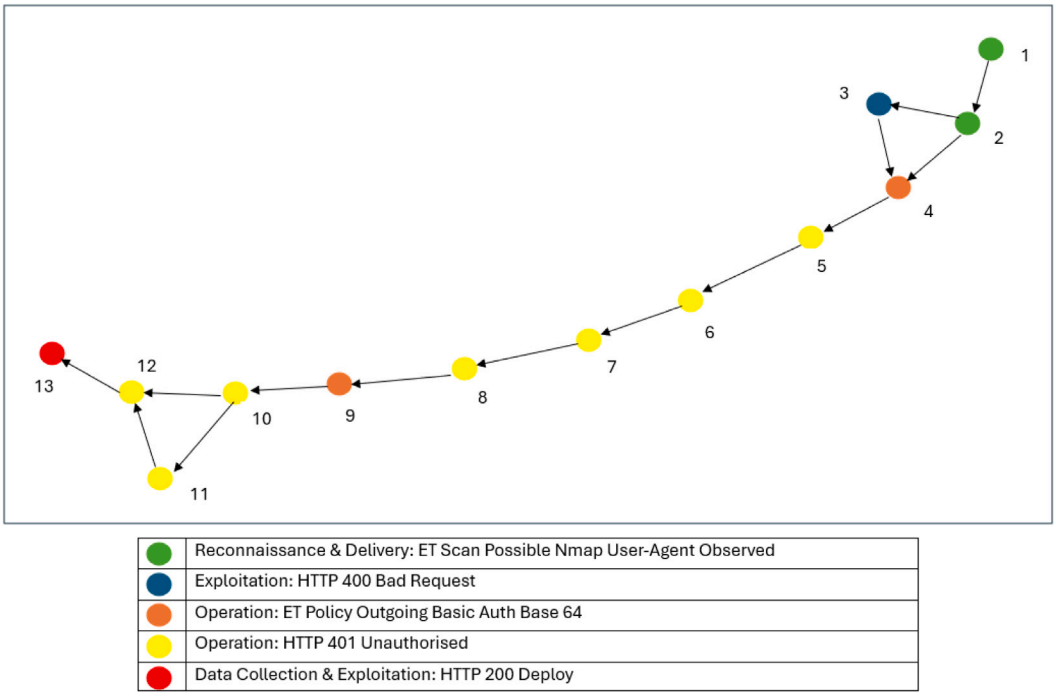

Fig. 5. Intrusion Kill Chain Categories for APT 28 Scenario test.

and specific details may be found within the paper focusing on the UWF-ZeekData24 dataset (Elam et al., 2025).

5. Evaluation methodology

In this section, the evaluation of the proposed method using the synthetic dataset and the ZeekData24 (Elam et al., 2025) publicly available real-world dataset is presented.

5.1. Evaluation on synthetic dataset

APTs are considered to have several stages of attack, known as the IKC (Mulligan, 2014). The IKC involve steps typical of an APT, with the target of attack and attack techniques from each stage forming the distinguishing characteristics of an APT over a typical attack. These steps include: Reconnaissance and Delivery, Exploitation, Operation, and Data Collection and Exfiltration. Reconnaissance involves data collection of materials involving resources such as employees and relationships to other institutions. The delivery process involves utilising an email or a type of malicious code derived from information gathered during the reconnaissance step, then sending via a legitimate internet server and attempting to convince users to access the malicious link. Exploitation involves the attacker planning an attack path for possible exploitation following the execution of destructive code from the victims end. The attacker then attempts to set up a Command and Control (C+C) centre for communication management on the victims device. Data Collection involves obtaining access permissions of target data through the aforementioned steps, along with manipulating settings and executing systems that can cause harm. Finally, exfiltration involves compiling and encoding the previously gathered information before transmitting it to predetermined servers. Simultaneously, the attacker erases any traces of their presence from the victim's system, making traceability of the APT difficult.

For the evaluation of the alert correlation method on APTs, a conceptual framework was developed using the aforementioned categories of meta alerts aligned with stages of the IKC. The primary objective is to reduce the volume of alerts to a more manageable level for security analysts to analyse and respond to attacks. As a result, accuracy was not the primary evaluation metric. This system is being developed in collaboration with an industrial partner and is being evaluated on real data, therefore with no labels available. Due to the confidentiality nature of the data, we cannot share details. At the same time, it is important to remark that our primary goal is not develop a system achieving high accuracy on a static labelled dataset, but a system employable to detect new attack patterns previously unknown. As such, rather than accuracy we prioritise the ability of the proposed method to reduce the amount of data that needs analysed by security analysts: i.e. reducing from hundreds of thousands of input records, to a few thousand records after aggregation, up to few hundred attack graphs after measuring the similarity between each pair of meta-alerts. For example, the industrial partner currently process a vast amount of data from different countries globally. If a cyber-attack were to take place in the United States, the correlation engine could be employed to aid in identifying and analysing the attack. The steps from the IKC are shown in Table 1. The synthetic data set described in Section 4.1 was used to map each meta alert to one of the IKC steps, providing insight into the current phase of the APT that the attackers have reached. An attack graph displays real-time attacks as correlated meta alerts, with information of the stage of the APT within the IKC, type of attack, victim IP address, attacker IP address, and port the attack is carried out in available for each node in the attack graph. An example using APT28 from the synthetic dataset for the evaluation of each of the steps follows.

Within the reconnnaissance and delivery stage, a port scan was carried out using NMAP to scan the host alongside ping scans. This was flagged by the IDS and labelled as an alert as 'ET SCAN Possible NMAP User-Agent Observed' and 'SURICATA ICMPv4 unknown code'. The next step of the IKC is Exploitation. This was carried out in APT28 with multiple attempts of password guessing and attempted file uploads resulting in multiple 'HTTP-400' alerts, shown in row A2 of Table 1. Operations such as password retrieval were detected by the IDS and labelled as 'ET Policy Outgoing Basic Auth' and 'Base64 HTTP password detected unencrypted' alerts and 'HTTP-401', shown on row A3 of Table 1. Finally, the last stage of the IKC process, data collection and exfiltration, were alerts flagged as 'HTTP-200 DEPLOY', shown within row A4 of Table 1. Fig. 5 shows the attack graph for APT28

with each of the IKC meta alert categories reflected on the nodes for each stage along the attack graph.

The hybrid response selection framework takes details of meta alerts from the correlation engine as an input to the hybrid response framework components. The alert ID, attack graph ID, meta alert category, IP source, country, destination name, destination port, type of attack, number of attacks and timestamp are used within each component to calculate a cost effective response to the current alert. Each of the components of the response framework, utilise past knowledge of similar attacks on the system, and responses previously deployed as a recommended response. The knowledge based system leverages system information and calculates a cost-sensitive response based on the impact and cost of the attack, and impact and cost of each of the responses available to deploy. A list of cost-sensitive responses are recommended as actions for response to the alerts.

The data-driven model leverages the probability of likelihood of attack calculated from the IDS, and produces a recommended response, based on the previous model outputs.

The recommendations from each of the response framework' s components undergo a test-driven verification process to ensure eligibility of responses for deployment, and finally, responses are prioritised for deployment. At the current stage of development, a HITL decides the final response to be deployed, however, over time, trust can be built on the learned recommended responses and the system can be automated for different levels of attacks, therefore deployed responses are stored to allow for future data mining.

5.2. Evaluation on ZeekData24 dataset

The final evaluation was conducted on the ZeekData24 (Elam et al., 2025). The dataset includes repeated attacks labelled using the MITRE ATT&CK framework (Strom et al., 2018). The dataset includes repeated attacks labelled using the MITRE ATT&CK framework. It contains several days of recordings; however, the days with attacks do not include benign data.

To better evaluate the efficacy of our approach in discriminating attacks from benign traffic, we applied an ad-hoc preprocessing strategy. The evaluation was conducted on 24-hour slices of the dataset. To mix benign and malicious traffic, each attack day was merged with data from a randomly selected day containing only benign traffic. The merge was performed by modifying the date of the benign records to match the date of the attack data, while preserving the original time. This ensured that benign background traffic was distributed across the 24-hour interval.

The resulting data were aggregated and processed using a correlation algorithm to generate attack graphs. Aggregation was based on source IP, destination IP, and destination port, grouping all records with matching IPs and ports that occurred within one hour of each other.

Given the nature of the dataset, we adopted an M2O approach, assuming a single attacker targeting multiple victims within the same network. The correlation weights were set as follows: source IP = 1.0, destination IP = 0.8, time = 0.2, destination port = 0.0, with an overall threshold of 0.87.

Since the goal was to identify multi-step attacks—presumably occurring across different ports and protocols—attack graphs resulting from the correlation were labelled as 'benign' if all events shared the same destination port, and as 'malicious' if they involved multiple destination ports. The metadata containing labels under the MITRE ATT&CK framework were used as ground truth for verification.

Benign records (labelled as 'none') that were included in attack graphs predicted as malicious were considered false positives. Conversely, attack-labelled samples that appeared in attack graphs predicted as benign were considered false negatives (see Table 2).

Table 3 presents the classification report for benign and attack graphs, calculated as the average across all attack days in the dataset.

Fig. 6 illustrates one of the malicious attack graphs extracted from the ZeekData24. The attacker IP (143.88.1.18) and the victims IP in the range 143.88.2.1–21 match the ones reported in the ZeekData24 report (Elam et al., 2025).

Table 2
Classification Report obtained on ZeekData24 dataset.

Class	Precision	Recall	F1-Score	Support
Benign	90.22%	68.23%	77.70%	2556
Malicious	87.04%	96.65%	91.60%	5644
Weighted Average	88.03%	87.79%	87.26%	8200
Macro-Average	88.63%	82.44%	84.65%	8200

Table 3
Data compression statistics.

Average Number ... per Day	Count	Percentage of reduction
of Alerts/Records	10,649	–
of meta-alerts	586	94.46%
of Attack Graphs	48	–
of Multi-Port/Suspicious Attack Graphs	20	57.95%

6. Correlation engine optimisation

The correlation engine processes extremely large quantities of data. The frequency of the correlation, consequently, depends on two main factors: (i) the velocity of the data being produced (how many records per hour/day), And (ii) the available computational resources. Data needs to be split into windows of observation large enough to capture attacks that might be intentionally spread over time by the attacker to go undetected. Consequently, the best results in terms of detection can be obtained with larger windows (hours or days of data) although the larger the window the larger is the amount of data to be processed that in introduces the limit of the approach due to the algorithmic complexity. In cases of low computational resources available the options become either using smaller time windows, or decreasing the frequency of the calculation, leaving enough time for the system to process the amount of input data. Maintaining the minimum memory requirement possible for the Correlation Engine algorithm is essential, as the system expects to handle large volumes of alerts. In order to optimise the current correlation engine algorithm, steps have been taken to achieve the best possible outcome for the specified requirements. The two streams of optimising the Correlation Engine algorithm are as follow:

1. Speed, resulting in faster execution. An updated algorithm performs larger batch computations using accelerated CPU vector functions available with the Numpy library.
2. Parallelisation, resulting in lower computation and faster execution. Adjusting the Correlation Engine algorithm to break the final matrix into smaller matrices to run in parallel, instead of generating one large sequence graph or several from iterating over the entire matrix.

Both streams combined result in reduction of the memory required for a single computation, which in turn uses that memory to facilitate faster computation of the result. More details on each of the optimisations are given below.

6.1. Speed

An optimised correlation engine algorithm was developed to utilise newly available memory space resulting from the parallelisation of the correlation engine, detailed in the next section. This updated algorithm performs larger batch computations using accelerated CPU vector functions from the Numpy library, achieved by recoding the algorithm to use Numpy matrices and operations instead of standard Python versions. For instance, the previous correlation engine version utilised time-based relationships in a one-by-one loop for each alert pair. The new approach establishes criteria for each relationship, evaluated on all alert pairs, storing intermediate matrices. These matrices then facilitate the application of logical case-based overrides to the

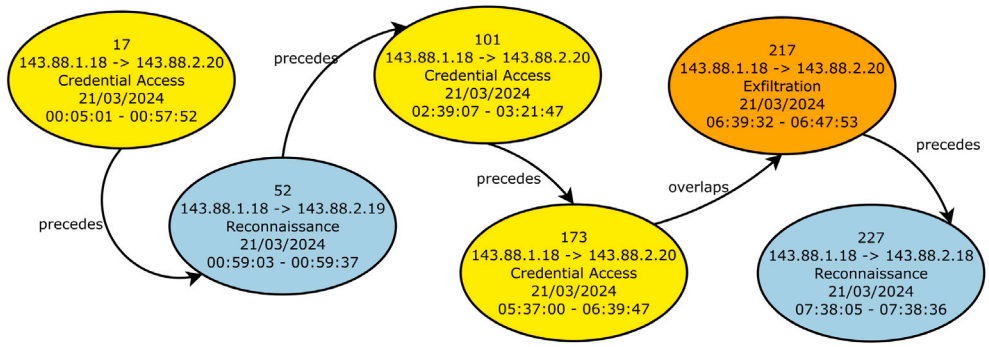

Fig. 6. One of the Malicious Attack Graph detected from the ZeekData24.

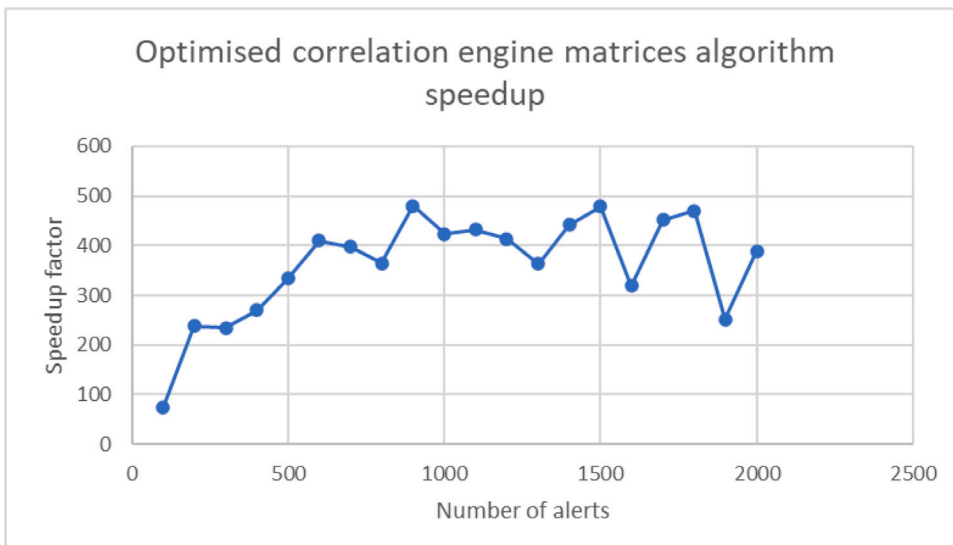

Fig. 7. Optimisaton of Corellation Engine Algorithm for Speed Up.

core time calculation. Despite the intermediate matrices being the same size as the resultant matrix (n2), the parallelisation, combined with the second optimisation, reduces the memory requirement of an individual computation segment. This optimisation exploits the saved memory to expedite result computation. Some matrices, like those based on string comparisons, cannot be readily optimised using Numpy, which focuses on numerical calculations. An example is the source organisation similarity, optimised with early-exit criteria to skip comparisons when safe, such as when the source organisation is unavailable. This optimisation accelerates the algorithm but introduces sensitivity to alert distribution, with speedup proportional to the number of alerts triggering the early-exit condition.

The algorithm updates demonstrate a speedup of 200+ times in computing matrices contributing to the similarity measure in example data and system-recorded live data over the last 30 days. The speedup specifically pertains to the calculation stage, not the full end-to-end pipeline. Comparative testing with matrix size suggests a relative speedup that increases with matrix size, plateauing between 300 and 400 times faster on average, as illustrated in Fig. 7.

Both algorithms exhibit a similar relative speed decrease as the number of alerts increases, however, the total time taken varies significantly. Virtual Memory usage for 500, 1000, and 2000 alerts indicates that the optimised algorithm has a "floor" memory requirement of 930MB for 500 rows, compared to the original algorithm's 460MB. The updated algorithm's memory usage is more stable than that of the original, as shown in Table 4.

Table 4
Virtual memory usage for original (v1) and updated (v2) algorithms.

Maximum virtual memory size		
Rows	v1	v2
500	463,820	932,084
1000	507,848	947,944
2000	671,932	925,220

A technical explanation for this stability could be attributed to recording memory at 0.1s intervals. The substantial speedup of the updated algorithm and the transient nature of new intermediate matrices suggest that 2000 alerts may not be sufficient to fully capture their impact at the chosen sampling rate.

6.2. Parallelisation

While a large matrix provides more than enough data to generate a full sequence graph, distances between alerts can grow very large and in such a case may indicate a decreased likelihood of a meaningful relationship that may be inferred. Depending on the scale of a matrix, the first and last alerts in a large enough matrix could occur months apart and ultimately prove unrelated. Splitting a single large matrix into smaller sub-matrices allows for more focused sequence graphs to be generated efficiently.

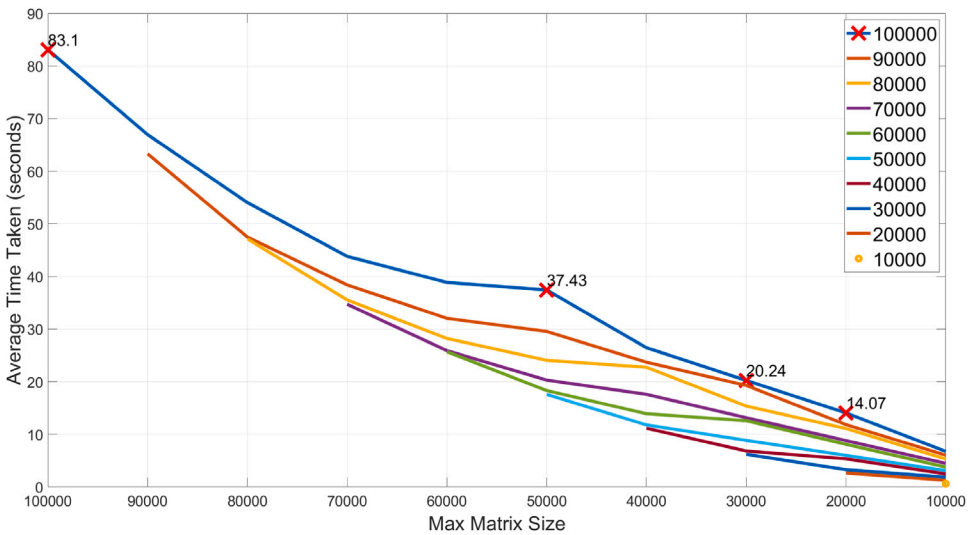

Fig. 8. Optimisation by Parallelisation.

Table 5
Difference in time to generate sequence graphs from matrix of 100,000 when splitting into matrices of max size (A) versus (B)

Matrix max size *A*	Matrix max size *B*	Difference in time taken *seconds–%*
100,000	50,000	−45.66−−54.95
100,000	30,000	−62.86−−75.64
100,000	20,000	−69.03−−83.07
100,000	50,000	−45.66−−54.95
50,000	30,000	−17.20−−45.93
50,000	20,000	−23.37−−62.42
30,000	20,000	−6.17−−30.49

To achieve this, a square matrix of size 100,000 is randomly generated from a seed. It was necessary to split the matrix into matrices of equal or lesser size where the size of the summed resultant matrices would equate to the initial matrix's size along only one axis. In short, the matrix was split from base sizes into smaller matrices of maximum sizes with any remainder forming a smaller matrix. Base sizes and maximum sizes ranged from 100,000 to 10,000 by reducing the value by 10% of 100,000 over each iteration.

These two ranges as sets form tuples of base and maximum sizes starting from (100,000 and 100,000) used to split the initial 100,000 matrix into a matrix of base size then further into matrices of maximum size. The second tuple would be (100,000 and 90,000), each subsequent tuple following this pattern of reducing the latter until 10,000 at which point the former would be reduced to 90,000 and the latter value reset to 100,000. Where a base matrix cannot be split into a larger maximum value, that iteration is represented as an empty value in the interest of its inclusion in results. Splitting includes values from the first value in each matrix so for reduction of base matrix the last 10% is removed.

Subsequent to matrix splitting, each resultant matrix is used to generate a sequence graph, the total time taken to generate all graphs in each instance of splitting are summed. This process was repeated 10 times and so the average total time to generate sequence graphs per instance is represented in Fig. 8 with select data points compared in Table 5.

Table 5 shows that time taken to generate sequence graphs decreases with each reduction in the maximum matrix size. Splitting the initial 100,000 matrix into 2 matrices of 50,000 results in reduction of time taken by half. While not a data point in Fig. 8, it can be estimated that a further division from 50,000 to 25,000 may indicate a similar decrease in time that is less than 50% of time taken for 50,000.

7. Discussion

The utilisation of meta alert stages in APT alerts introduces a streamlined approach to prioritise attack scenarios. This involves presenting the attack graph visually with tool-tips indicating the meta alert stage alongside other useful attack information. By examining the attack graph and the information embedded in each node, the current stage of an APT can be easily identified in a live attack. This method simplifies the prioritisation of attacks through visualisation, assigning higher priority to either the longest attack graph or the attack graph with nodes falling into the highest number of different meta alert categories. For example, consider the synthetic dataset example in Fig. 5. In this context, the largest attack graph would be prioritised over the smaller attack graphs, since the latter comprises only five attacks, and three attacks, all categorised as reconnaissance and delivery, the earliest stage of the APT. In contrast, APT28's attack graph encompasses a greater number of nodes and a greater number of meta alert categories reached throughout the attack, indicating a more advanced stage in the APT progression, making it deserving of higher priority. Furthermore, the categorisation of meta alerts also offers a further layer of granularity to the components of the hybrid response framework, where the stage of the APT offers more information on an attack priority, and can be learned accordingly in the knowledge-based and data-driven learning systems.

The evaluation on the ZeekData24 dataset highlighted the strong potential of the proposed approach. In particular, the recall for the malicious class reached 96.65%. This high recall comes at the expense of the benign class, which experienced some false positives.

However, considering the risks associated with cyber threats, achieving higher recall for the malicious class is preferable to minimise the likelihood of attacks going undetected.

Moreover, compressing raw alerts into meta-alerts enables security analysts to focus on potentially malicious activity rather than benign events, while also reducing the overall volume of alerts to manage. For example, as shown in Table 3, 10,649 alerts can be compressed into 586 meta-alerts, further reduced to 48 attack graphs of which 20 are suspected malicious AG. This level of compression represents a substantial improvement in supporting analysts when responding to large scale APT threats.

8. Conclusion

This paper introduces advancements in developing a robust real-time correlation engine designed for identifying APTs within an enterprise network. Throughout the paper, four key contributions were

presented to enhance the research in intrusion detection and response systems. Firstly, a novel alert correlation method for APTs was described. Next, a novel dataset encompassing various APT scenarios was generated to assess the correlation engine's performance. The conceptual framework was evaluated by categorising meta-alerts based on the APT stage reached by the attacker, with the visualisation of the attack priorities displayed on a dashboard for the analysts to examine. This paper also discussed the identified shortcomings of the correlation engine and provided preliminary results of optimisation efforts implemented on the system. Finally, a hybrid knowledge and data-driven response system was proposed, aiming to offer timely response recommendations by utilising meta alert information as input to the response framework components. Future work will concentrate on refining the correlation engine within enterprise networks. The focus will be on real-time distribution of meta alerts across the correlation engine and modelling the potential impact of a deployed response throughout these networks. This approach will enable analysts to comprehend the consequences of a response action on the system before execution.

CRediT authorship contribution statement

Bronagh Lanigan: Conceptualization, Formal analysis, Investigation, Methodology, Software, Writing – original draft. **Zeinab Rezaeifar:** Conceptualization, Formal analysis, Investigation, Methodology, Software, Writing – original draft. **Federico Cruciani:** Conceptualization, Formal analysis, Investigation, Methodology, Software, Writing – original draft. **Michael Milliken:** Conceptualization, Formal analysis, Investigation, Methodology, Software, Writing – original draft. **Jordan Vincent:** Conceptualization, Formal analysis, Investigation, Methodology, Software, Writing – original draft. **Samuel Moore:** Conceptualization, Software, Writing – review & editing. **Muhammad Aaqib:** Methodology, Writing – review & editing. **Alan Mills:** Software, Writing – review & editing. **Pushpinder K. Chouhan:** Conceptualization, Methodology, Writing – review & editing. **Alfie Beard:** Conceptualization, Supervision, Writing – review & editing. **Chris D. Nugent:** Conceptualization, Supervision, Writing – review & editing. **Luke Chen:** Conceptualization, Supervision, Writing – review & editing. **Alex Healing:** Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research is supported by the BT Ireland Innovation Centre (BTIIC) project, funded by British Telecom, UK and Invest Northern Ireland, UK.

Data availability

Data will be made available on request.

References

Alserhani, F. M. (2016). Alert correlation and aggregation techniques for reduction of security alerts and detection of multistage attack. *International Journal of Advanced Studies in Computers, Science and Engineering*, *5*(2).

Alsubhi, K., Al-Shaer, E., & Boutaba, R. (2008). Alert prioritization in intrusion detection systems. In *NOMS 2008-2008 IEEE network operations and management symposium* (pp. 33–40). IEEE.

Andrew, J., & Kathrine, G. J. W. (2018). An intrusion detection system using correlation, prioritization and clustering techniques to mitigate false alerts. In *Advances in big data and cloud computing* (pp. 257–268). Springer.

Anuar, N. B., Papadaki, M., Furnell, S., & Clarke, N. (2010). An investigation and survey of response options for intrusion response systems (IRSs). In *2010 information security for South africa* (pp. 1–8). http://dx.doi.org/10.1109/ISSA.2010.5588654.

Assante, M. J., & Lee, R. M. (2015). The industrial control system cyber kill chain. *SANS Institute InfoSec Reading Room*, *1*, 24.

Barzegar, M., & Shajari, M. (2018). Attack scenario reconstruction using intrusion semantics. *Expert Systems with Applications*, *108*, http://dx.doi.org/10.1016/j.eswa.2018.04.030.

Bateni, M., Baraani, A., Ghorbani, A., & Rezaei, A. (2013). An ais-inspired architecture for alert correlation. *International Journal of Innovative Computing, Information and Control*, *9*(1).

Chouhan, P. K., Quigley, B., Beard, A., & Chen, L. (2021). A cost based dynamic response method for internet of things cyberattacks. In *2021 9th international symposium on digital forensics and security* (pp. 1–6). http://dx.doi.org/10.1109/ISDFS52919.2021.9486362.

Chouhan, P. K., Quigley, B., Beard, A., & Chen, L. (2023). A test-driven action verification method for intrusion response systems. In *2023 IEEE international conference on metaverse computing, networking and applications* (pp. 114–118). http://dx.doi.org/10.1109/MetaCom57706.2023.00032.

Cruciani, F., Moore, S., Quigley, B., Nugent, C., & Sani, S. (2022). Semi-supervised detection of algorithmically generated domains using neural network-based autoencoders.

Cuppens, F., & Ortalo, R. (2000). LAMBDA: A language to model a database for detection of attacks. *Vol. 1907*, In *Lecture notes in computer science (including subseries lecture notes in artificial intelligence and lecture notes in bioinformatics)*. http://dx.doi.org/10.1007/3-540-39945-3_13.

Do Xuan, C., & Dao, M. H. (2021). A novel approach for APT attack detection based on combined deep learning model. *Neural Computing and Applications*, *33*, 13251–13264.

Eckmann, S. T., Vigna, G., & Kemmerer, R. A. (2002). STATL: An attack language for state-based intrusion detection. *Journal of Computer Security*, *10*(1–2), 71–103. http://dx.doi.org/10.3233/JCS-2002-101-204.

Elam, M., Mink, D., Bagui, S. S., Plenkers, R., & Bagui, S. C. (2025). Introducing UWF-ZeekData24: An enterprise MITRE ATT&CK labeled network attack traffic dataset for machine learning/AI. *Data*, *10*(5), http://dx.doi.org/10.3390/data10050059, URL https://www.mdpi.com/2306-5729/10/5/59.

Elshoush, H. T., & Osman, I. M. (2011). Alert correlation in collaborative intelligent intrusion detection systems - a survey. *Applied Soft Computing Journal*, *11*(7), http://dx.doi.org/10.1016/j.asoc.2010.12.004.

Godefroy, E., Totel, E., Hurfin, M., & Majorczyk, F. (2014). Automatic generation of correlation rules to detect complex attack scenarios. In *2014 10th international conference on information assurance and security, IAS 2014*. http://dx.doi.org/10.1109/ISIAS.2014.7064615.

González-Granadillo, G., González-Zarzosa, S., & Diaz, R. (2021). Security information and event management (SIEM): analysis, trends, and usage in critical infrastructures. *Sensors*, *21*(14), 4759.

Gupta, N., Jindal, V., & Bedi, P. (2023). A survey on intrusion detection and prevention systems. *SN Computer Science*, *4*, 439. http://dx.doi.org/10.1007/s42979-023-01926-7.

Haas, S., & Fischer, M. (2019). On the alert correlation process for the detection of multi-step attacks and a graph-based realization. *ACM SIGAPP Applied Computing Review*, *19*(1), http://dx.doi.org/10.1145/3325061.3325062.

Hassan, W. U., Bates, A., & Marino, D. (2020). Tactical provenance analysis for endpoint detection and response systems. In *2020 IEEE symposium on security and privacy* (pp. 1172–1189). IEEE.

Hersén, N., Hacks, S., & Fögen, K. (2021). Towards measuring test coverage of attack simulations. In A. Augusto, A. Gill, S. Nurcan, I. Reinhartz-Berger, R. Schmidt, & J. Zdravkovic (Eds.), *Enterprise, business-process and information systems modeling* (pp. 303–317). Cham: Springer International Publishing.

Hu, H., Liu, J., Zhang, Y., Liu, Y., Xu, X., & Tan, J. (2020). Attack scenario reconstruction approach using attack graph and alert data mining. *Journal of Information Security and Applications*, *54*, Article 102522.

Hussain, T., Beard, A., Chen, L., Nugent, C., Liu, J., & Moore, A. (2022). From machine learning based intrusion detection to cost sensitive intrusion response. In *2022 6th international conference on cryptography, security and privacy* (pp. 124–130). http://dx.doi.org/10.1109/CSP55486.2022.00031.

Iannucci, S., & Abdelwahed, S. (2018). Model-based response planning strategies for autonomic intrusion protection. *ACM Trans. Auton. Adapt. Syst.*, *13*(1), http://dx.doi.org/10.1145/3168446.

Iannucci, S., Barba, O. D., Cardellini, V., & Banicescu, I. (2019). A performance evaluation of deep reinforcement learning for model-based intrusion response. In *2019 IEEE 4th international workshops on foundations and applications of self* systems* (pp. 158–163). URL https://api.semanticscholar.org/CorpusID:199509673.

Inayat, Z., Gani, A., Anuar, N. B., Khan, M. K., & Anwar, S. (2016). Intrusion response systems: Foundations, design, and challenges. *Journal of Network and Computer Applications*, *62*, 53–74. http://dx.doi.org/10.1016/j.jnca.2015.12.006, URL https://www.sciencedirect.com/science/article/pii/S1084804515002994.

Irshad, H., Ciocarlie, G., Gehani, A., Yegneswaran, V., Lee, K. H., Patel, J., et al. (2021). Trace: Enterprise-wide provenance tracking for real-time apt detection. *IEEE Transactions on Information Forensics and Security*, *16*, 4363–4376.

Julisch, K. (2001). Mining alarm clusters to improve alarm handling efficiency. *Vol. 2001-January*, In *Proceedings - annual computer security applications conference, ACSAC* (pp. 12–21). IEEE Computer Society, http://dx.doi.org/10.1109/ACSAC.2001.991517.

Kholidy, H. A. (2021). Autonomous mitigation of cyber risks in the cyber–physical systems. *Future Generation Computer Systems*, *115*, 171–187. http://dx.doi.org/10.1016/j.future.2020.09.002, URL https://www.sciencedirect.com/science/article/pii/S0167739X19320680.

Kholidy, H. A., Erradi, A., Abdelwahed, S., & Baiardi, F. (2016). A risk mitigation approach for autonomous cloud intrusion response system. *Computing*, *98*, 1111–1135, URL https://api.semanticscholar.org/CorpusID:243957.

Khosravi, M., & Ladani, B. T. (2020). Alerts correlation and causal analysis for APT based cyber attack detection. *IEEE Access*, *8*, 162642–162656.

Kocsis, L., & Szepesvári, C. (2006). Bandit based monte-carlo planning. In *European conference on machine learning* (pp. 282–293). Springer.

Lanoe, D., Hurfin, M., & Totel, E. (2019). A scalable and efficient correlation engine to detect multi-step attacks in distributed systems. *Vol. 2019-October*, In *Proceedings of the IEEE symposium on reliable distributed systems*. http://dx.doi.org/10.1109/SRDS.2018.00014.

Lv, Y., Qin, S., Zhu, Z., Yu, Z., Li, S., & Han, W. (2022). A review of provenance graph based APT attack detection: Applications and developments. In *2022 7th IEEE international conference on data science in cyberspace* (pp. 498–505). IEEE.

Mateos, V., Villagrá, V. A., Romero, F., & Berrocal, J. Definition of response metrics for an ontology-based automated intrusion response systems. *Computers & Electrical Engineering*, *38*(5), 1102–1114.

Meng, W., Li, W., & Kwok, L.-F. (2015). Design of intelligent KNN-based alarm filter using knowledge-based alert verification in intrusion detection. *Security and Communication Networks*, *8*(18), 3883–3895.

Meng, Y., Li, W., et al. (2012). Intelligent alarm filter using knowledge-based alert verification in network intrusion detection. In *International symposium on methodologies for intelligent systems* (pp. 115–124). Springer, http://dx.doi.org/10.1007/978-3-642-34624-8_14.

Mirheidari, S. A., Arshad, S., & Jalili, R. (2013). *Lecture notes in computer science (including subseries lecture notes in artificial intelligence and lecture notes in bioinformatics)*: *Vol. 8300 LNCS*, *Alert correlation algorithms: A survey and taxonomy*. http://dx.doi.org/10.1007/978-3-319-03584-0_14.

Moore, S. J., Cruciani, F., Nugent, C. D., Zhang, S., Cleland, I., & Sani, S. (2023). Deep learning for network intrusion: A hierarchical approach to reduce false alarms. *Intelligent Systems with Applications*, *18*, Article 200215. http://dx.doi.org/10.1016/j.iswa.2023.200215, URL https://www.sciencedirect.com/science/article/pii/S2667305323000406.

Mulligan, J. J. (2014). A "kill chain" analysis of the 2013 target dtaa breach. In *The Target Store Data Breaches: Examination and Insight*.

Navarro, J., Deruyver, A., & Parrend, P. (2018). A systematic survey on multi-step attack detection. *Vol. 76*, In *Computers and security*. http://dx.doi.org/10.1016/j.cose.2018.03.001.

Nejat, S. K., & Kabiri, P. (2017). An adaptive and cost-based intrusion response system. *Cybernetics and Systems*, *48*(6–7), 495–509. http://dx.doi.org/10.1080/01969722.2017.1319693.

Neuschmied, H., Winter, M., Stojanović, B., Hofer-Schmitz, K., Božić, J., & Kleb, U. (2022). Apt-attack detection based on multi-stage autoencoders. *Applied Sciences*, *12*(13), 6816.

Ning, P., Cui, Y., & Reeves, D. S. (2002). Constructing attack scenarios through correlation of intrusion alerts. In *Proceedings of the ACM conference on computer and communications security*. http://dx.doi.org/10.1145/586143.586144.

Niu, W., Zhou, J., Zhao, Y., Zhang, X., Peng, Y., & Huang, C. (2022). Uncovering APT malware traffic using deep learning combined with time sequence and association analysis. *Computers & Security*, *120*, Article 102809.

Ou, X., Govindavajhala, S., Appel, A. W., et al. (2005). MulVAL: A logic-based network security analyzer. *Vol. 8*, In *USeNIX security symposium* (pp. 113–128). Baltimore, MD.

Park, N.-E., Lee, Y.-R., Joo, S., Kim, S.-Y., Kim, S.-H., Park, J.-Y., et al. (2023). Performance evaluation of a fast and efficient intrusion detection framework for advanced persistent threat-based cyberattacks. *Computers & Electrical Engineering*, *105*, Article 108548.

Qi, Y., Gu, Z., Li, A., Zhang, X., Shafiq, M., Mei, Y., et al. (2023). Cybersecurity knowledge graph enabled attack chain detection for cyber-physical systems. *Computers & Electrical Engineering*, *108*, Article 108660. http://dx.doi.org/10.1016/j.compeleceng.2023.108660, URL https://www.sciencedirect.com/science/article/pii/S004579062300085X.

Qin, X., & Lee, W. (2004). Attack plan recognition and prediction using causal networks. In *Proceedings - annual computer security applications conference, ACSAC*. http://dx.doi.org/10.1109/CSAC.2004.7.

Ramaki, A. A., Amini, M., & Ebrahimi Atani, R. (2015). RTECA: Real time episode correlation algorithm for multi-step attack scenarios detection. *Computers and Security*, *49*, http://dx.doi.org/10.1016/j.cose.2014.10.006.

Ren, H., Stakhanova, N., & Ghorbani, A. A. (2010). *Lecture notes in computer science (including subseries lecture notes in artificial intelligence and lecture notes in bioinformatics)*: *Vol. 6201 LNCS*, *An online adaptive approach to alert correlation*. http://dx.doi.org/10.1007/978-3-642-14215-4_9.

Saad, S., & Traore, I. (2013). Semantic aware attack scenarios reconstruction. *Journal of Information Security and Applications*, *18*(1), http://dx.doi.org/10.1016/j.jisa.2013.08.002.

Sharma, A., Gupta, B. B., Singh, A. K., & Saraswat, V. (2023). A novel approach for detection of APT malware using multi-dimensional hybrid Bayesian belief network. *International Journal of Information Security*, *22*(1), 119–135.

Shen, F., Perigo, L., Curry, J. H., et al. (2023). SR2APT: A detection and strategic alert response model against multistage APT attacks. *Security and Communication Networks*, *2023*.

Shittu, R., Healing, A., Ghanea-Hercock, R., Bloomfield, R., & Muttukrishnan, R. (2014). OutMet: A new metric for prioritising intrusion alerts using correlation and outlier analysis. In *Proceedings - conference on local computer networks, LCN*. http://dx.doi.org/10.1109/LCN.2014.6925787.

Shittu, R., Healing, A., Ghanea-Hercock, R., Bloomfield, R., & Rajarajan, M. (2015). Intrusion alert prioritisation and attack detection using post-correlation analysis. *Computers and Security*, *50*, http://dx.doi.org/10.1016/j.cose.2014.12.003.

Strom, B. E., Applebaum, A., Miller, D. P., Nickels, K. C., Pennington, A. G., & Thomas, C. B. (2018). *Mitre att&ck: Design and philosophy*: *Technical report*, The MITRE Corporation.

Wang, Y., Guo, Y., & Fang, C. (2022). An end-to-end method for advanced persistent threats reconstruction in large-scale networks based on alert and log correlation. *Journal of Information Security and Applications*, *71*, Article 103373.

Xiang, G., Dong, X., & Yu, G. (2005). Correlating alerts with a data mining based approach. In *Proceedings - 2005 IEEE international conference on e-technology, e-commerce and e-service, EEE-05* (pp. 341–346). http://dx.doi.org/10.1109/eee.2005.56.

Xiong, C., Zhu, T., Dong, W., Ruan, L., Yang, R., Cheng, Y., et al. (2022). <Sc>conan</sc>: A practical real-time APT detection system with high accuracy and efficiency. *IEEE Transactions on Dependable and Secure Computing*, *19*(1), 551–565. http://dx.doi.org/10.1109/TDSC.2020.2971484.

Xosanavongsa, C., Totel, E., & Bettan, O. (2019a). Discovering correlations: A formal definition of causal dependency among heterogeneous events. In *2019 IEEE European symposium on security and privacy* (pp. 340–355). IEEE.

Xosanavongsa, C., Totel, E., & Bettan, O. (2019b). Discovering correlations: A formal definition of causal dependency among heterogeneous events. In *Proceedings - 4th IEEE European symposium on security and privacy, EURO s and p 2019*. http://dx.doi.org/10.1109/EuroSP.2019.00033.

Zhou, C. V., Leckie, C., & Karunasekera, S. (2010). A survey of coordinated attacks and collaborative intrusion detection. *Computers & Security*, *29*(1), 124–140.

Zhu, B., & Ghorbani, A. A. (2006). Alert correlation for extracting attack strategies. *International Journal of Network Security*, *3*(3).

Zimba, A., Chen, H., Wang, Z., & Chishimba, M. (2020). Modeling and detection of the multi-stages of advanced persistent threats attacks based on semi-supervised learning and complex networks characteristics. *Future Generation Computer Systems*, *106*, 501–517.